

NYCU IAL/DL 2026 LLM-1：SFT on Answer-only Data 實驗報告

1. 任務簡介

本次作業目標為針對單選題資料進行大語言模型的 Supervised Fine-tuning (SFT)，使模型能夠根據題目與選項，輸出正確答案 `A`、`B`、`C` 或 `D`。競賽要求只能使用中國釋出的模型，例如 Qwen、QwQ、DeepSeek 系列等，並且需在本地端機器上完成模型訓練與推論，不可使用雲端大型語言模型產生測試集答案。

本實驗主要使用 `Qwen2.5-Instruct` 系列模型作為 base model，並採用 LoRA / QLoRA 的方式進行參數高效微調。由於任務輸出為單選題答案，因此推論時不採用自由生成長文本，而是直接比較模型在答案位置對 `A/B/C/D` 四個 token 的 logit 分數，選出最高者作為最終答案。

本實驗的核心目標包括：

1. 建立可在本地端執行的 SFT 訓練流程。
2. 將原始 CSV 題目資料整理為 prompt-answer 格式。
3. 使用 answer-only loss，使模型只針對答案部分進行學習。
4. 比較不同模型大小、資料組合與訓練參數對 Kaggle score 的影響。
5. 分析 validation accuracy 與 Kaggle score 不一致的可能原因。

2. 使用資料

本次實驗主要使用以下資料：

資料檔案	用途
<code>train.csv</code>	官方訓練資料
<code>test-check-v2.csv</code>	Kaggle 測試資料
<code>sample_submission.csv</code>	Kaggle 提交格式範本
<code>HW1_.csv</code>	額外補充訓練資料

在實驗中，資料分為兩種設定：

2.1 Official data

僅使用官方提供的 `train.csv` 作為訓練資料。這種設定的優點是資料分布較接近官方測試集，因此可能具有較好的泛化能力。

2.2 Mixed data

將官方 `train.csv` 與額外補充資料 `HW1_.csv` 合併後進行訓練。此設定可以增加訓練資料量，但實驗後發現，補充資料的答案分布可能與官方資料不同，尤其答案 `B` 偏多、`D` 偏少，導致模型產生輸出偏差，Kaggle score 並未提升。

3. 技術方法

3.1 Supervised Fine-tuning, SFT

本作業使用 supervised fine-tuning 的方式訓練模型。對於每一筆選擇題資料，將題目與選項組成 prompt，正確答案 `A/B/C/D` 作為 completion。

資料格式概念如下：

請根據以下題目與選項，選出唯一正確答案。
規則：只輸出 A、B、C 或 D，不要輸出解釋。

題目：...

A. ...

B. ...

C. ...

D. ...

答案：

模型的目標輸出為：

A

或 `B/C/D`。

由於此任務只需要模型回答選項，因此訓練時不要求模型學習完整的推理過程，而是讓模型學會在看到題目與選項後，直接輸出正確答案。

3.2 Answer-only loss

本作業的重點是 `answer-only data`，因此模型訓練時不應該把 prompt 內容也納入 loss 計算。若將 prompt 也納入 loss，模型會花費能力去學習題目格式，而不是專注於答案判斷。

因此在資料處理時，程式會將 prompt 對應的 label 設為 `-100`，使 PyTorch 的 cross-entropy loss 忽略這些位置，只計算答案 token 的 loss。

概念如下：

```
input_ids: [prompt tokens ... answer token]
labels:    [-100, -100, ... A]
```

如此一來，模型只會被訓練成在 `答案：` 之後預測正確的 `A/B/C/D`。

3.3 LoRA / QLoRA

由於完整微調大型語言模型需要大量 GPU 記憶體，本實驗採用 LoRA / QLoRA 進行參數高效微調。

LoRA 的核心概念是凍結原本的大模型權重，只在模型特定線性層旁邊加入低秩矩陣作為可訓練參數。因此訓練時只需更新少量 adapter 權重，可以大幅降低 GPU 記憶體需求。

本實驗中主要調整的 LoRA 參數包括：

參數	說明
<code>lora_r</code>	LoRA 低秩矩陣的 rank，控制 adapter 容量
<code>lora_alpha</code>	LoRA scaling factor，通常與 <code>lora_r</code> 搭配設定
<code>lora_dropout</code>	LoRA 訓練時的 dropout，用於降低過擬合

本實驗最佳組合為：

```
learning rate = 5e-5
lora_r = 16
lora_dropout = 0.10
```

此設定代表模型使用中等容量的 LoRA adapter，並以較低 learning rate 進行穩定更新，同時使用 10% dropout 減少過擬合。

3.4 Logit-based inference

由於本任務是單選題，最終輸出只需要 `A/B/C/D`，因此推論階段沒有使用自由文字生成，而是採用 logit scoring。

具體做法是：

1. 將測試題目整理成與訓練時相同的 prompt。
2. 將 prompt 輸入模型。
3. 取得模型在下一個 token 位置對 `A`、`B`、`C`、`D` 的 logit。
4. 選擇 logit 最大的選項作為答案。

這種方式相較於自由生成有以下優點：

1. 可以避免模型輸出多餘解釋。

2. 可以確保 submission 欄位只包含 `A/B/C/D`。
3. 對選擇題任務更穩定。
4. 避免因 decoding strategy 造成格式錯誤。

4. 程式碼架構與說明

本專案主要包含四個核心程式：

```
scripts/  
├─ prepare_data.py  
├─ train_qlora_sft.py  
├─ quick_eval.py  
└─ infer_submission.py
```

整體流程如下：

```
原始 CSV  
↓  
prepare_data.py  
↓  
train.jsonl / valid.jsonl  
↓  
train_qlora_sft.py  
↓  
LoRA adapter  
↓  
quick_eval.py  
↓  
validation accuracy  
↓  
infer_submission.py  
↓  
submission.csv
```

4.1 `prepare_data.py`

`prepare_data.py` 負責將原始 CSV 檔案整理成模型可訓練的 JSONL 格式。主要功能包括：

1. 讀取官方訓練資料 `train.csv`。
2. 視實驗設定選擇是否讀取補充資料 `HW1_.csv`。
3. 清理題目與選項文字。
4. 統一答案格式，例如將 `A`、`A.` 等格式整理為 `A`。
5. 移除不合法答案，例如非 `A/B/C/D` 的資料。
6. 將題目組成 prompt，將答案作為 completion。

7. 切分 train / validation set。
8. 輸出 `train.jsonl`、`valid.jsonl` 與資料整理報告。

輸出檔案如下：

輸出檔案	說明
<code>train.jsonl</code>	訓練資料
<code>valid.jsonl</code>	本地驗證資料
<code>cleaned_sft_data.csv</code>	清理後的完整資料
<code>prepare_report.json</code>	資料整理統計資訊

此程式是整個訓練流程的第一步，若沒有先執行，後續 `train_qlora_sft.py` 將找不到 `train.jsonl` 與 `valid.jsonl`。

4.2 `train_qlora_sft.py`

`train_qlora_sft.py` 是主要訓練程式。其功能包括：

1. 載入指定的 Qwen base model。
2. 使用 4-bit quantization 降低 GPU 記憶體需求。
3. 套用 LoRA adapter。
4. 讀取 `train.jsonl` 與 `valid.jsonl`。
5. 建立 answer-only loss 的訓練資料。
6. 進行 SFT 訓練。
7. 儲存訓練完成的 LoRA adapter。

其中最重要的是 answer-only loss 的處理。程式會將 prompt 部分的 labels 設為 `-100`，使 loss 只作用於答案 token。

概念如下：

```
labels[:prompt_len] = [-100] * prompt_len
```

這代表模型在訓練時只需要學會輸出正確答案，不需要學習重建題目或選項內容。

訓練時常用參數包括：

參數	說明
<code>--model_name</code>	使用的 base model，例如 <code>Qwen/Qwen2.5-0.5B-Instruct</code>
<code>--train_jsonl</code>	訓練資料路徑
<code>--valid_jsonl</code>	驗證資料路徑
<code>--output_dir</code>	LoRA adapter 輸出資料夾

參數	說明
<code>--epochs</code>	訓練 epoch 數
<code>--max_length</code>	最大 token 長度
<code>--lr</code>	learning rate
<code>--batch_size</code>	單次 batch size
<code>--grad_accum</code>	gradient accumulation steps
<code>--lora_r</code>	LoRA rank
<code>--lora_alpha</code>	LoRA scaling factor
<code>--lora_dropout</code>	LoRA dropout

4.3 `quick_eval.py`

`quick_eval.py` 用來在本地 validation set 上快速評估模型表現。其功能為：

1. 載入 base model。
2. 載入訓練好的 LoRA adapter。
3. 讀取 `valid.jsonl`。
4. 對每一題計算 `A/B/C/D` 的 logit。
5. 選出 logit 最高的選項。
6. 與 validation label 比較，計算 validation accuracy。

需要注意的是，validation accuracy 只代表模型在本地切分資料上的表現，並不等於 Kaggle score。實驗中可以觀察到，部分模型 validation accuracy 很高，但 Kaggle score 反而降低，這代表模型可能對本地 validation split 過擬合，或 validation set 與測試集分布不一致。

4.4 `infer_submission.py`

`infer_submission.py` 負責產生 Kaggle submission 檔案。其功能包括：

1. 載入 base model 與 LoRA adapter。
2. 讀取 `test-check-v2.csv`。
3. 依照訓練時相同格式建立 prompt。
4. 使用 logit-based inference 選出 `A/B/C/D`。
5. 讀取 `sample_submission.csv` 保持正確格式。
6. 輸出 `submission.csv`。

本實驗中所有 submission 皆使用 `logit` 推論方式，避免模型自由生成多餘文字。

5. 訓練環境與執行流程

本實驗於本地端 VS Code + Miniconda 環境執行，並使用 NVIDIA GPU 進行訓練。

5.1 環境建立

建立 conda 環境：

```
conda create -n nycu-llm1 python=3.10 -y
conda activate nycu-llm1
```

安裝 PyTorch、Transformers、PEFT、Datasets、BitsAndBytes 等套件。

實際執行時曾遇到 `transformers` 與 `torch` 版本相容問題，因此後續將 `transformers` 降到較穩定版本，以避免匯入錯誤。

5.2 GPU 確認

執行以下指令確認 GPU 可用：

```
python -c "import torch; print(torch.cuda.is_available()); print(torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'CPU only')"
```

確認結果為：

```
True
NVIDIA GeForce RTX 3060 Laptop GPU
```

5.3 OpenMP 問題處理

在 Windows 環境中，執行 PyTorch 相關程式時曾遇到 OpenMP runtime 重複載入問題，因此在 PowerShell 中加入：

```
$env:KMP_DUPLICATE_LIB_OK="TRUE"
```

此設定可暫時避免 `libiomp5md.dll` 重複初始化造成程式中斷。

5.4 訓練流程

典型訓練流程如下：

```
python scripts/prepare_data.py `
  --train_csv raw_data/train.csv `
  --out_dir data_official `
  --valid_ratio 0.1
```

接著執行模型訓練：

```
python -u scripts/train_qlora_sft.py `
--model_name Qwen/Qwen2.5-0.5B-Instruct `
--train_jsonl data_official/train.jsonl `
--valid_jsonl data_official/valid.jsonl `
--output_dir outputs/qwen25_05b_official_ep3_lr5e5_r16_drop10 `
--epochs 3 `
--max_length 768 `
--lr 5e-5 `
--batch_size 1 `
--grad_accum 16 `
--lora_r 16 `
--lora_alpha 32 `
--lora_dropout 0.10 `
--warmup_ratio 0.05
```

最後產生 Kaggle submission :

```
python -u scripts/infer_submission.py `
--base_model Qwen/Qwen2.5-0.5B-Instruct `
--adapter_path outputs/qwen25_05b_official_ep3_lr5e5_r16_drop10 `
--test_csv raw_data/kaggle_test_set_792.csv `
--sample_submission raw_data/kaggle_submission_template_792.csv `
--output_csv outputs/submission.csv `
--method logit
```

6. 實驗設定與結果

本次實驗共測試五組主要設定，包含不同模型大小、資料組合、epoch 數與訓練參數。

實驗	模型	資料	Epoch	推論方式	Validation Accuracy	Kaggle Score
Exp-1	Qwen2.5-0.5B-Instruct	official	1	logit	0.8462	0.62127
Exp-2	Qwen2.5-1.5B-Instruct	official	3	logit	0.9873	0.61276
Exp-3	Qwen2.5-0.5B-Instruct	official	3	logit	0.9223	0.66382
Exp-4	Qwen2.5-0.5B-Instruct	mixed	3	logit	0.8864	0.63404
Exp-5	Qwen2.5-0.5B-Instruct	official	4	logit	0.9223	0.63829

7. 實驗分析

7.1 Exp-1 : 0.5B official, epoch 1

Exp-1 使用 `Qwen2.5-0.5B-Instruct`，並僅以官方訓練資料訓練 1 個 epoch。此組為 baseline 實驗。

結果：

```
validation accuracy: 0.8462
Kaggle score: 0.62127
```

此結果顯示，即使只訓練 1 個 epoch，模型已明顯高於隨機猜測，代表 answer-only SFT 與 logit 推論方式能夠有效讓模型學習單選題任務。

7.2 Exp-2 : 改用 1.5B 模型並增加訓練量

Exp-2 將模型從 0.5B 改為 `Qwen2.5-1.5B-Instruct`，並將 epoch 增加為 3，希望透過更大的模型容量提升表現。

結果：

```
validation accuracy: 0.9873
Kaggle score: 0.61276
```

雖然 validation accuracy 顯著提高，但 Kaggle score 反而低於 Exp-1。此現象代表模型可能對本地 validation split 過擬合。由於官方訓練資料量不大，1.5B 模型容量較高，若訓練 epoch 過多，容易記住訓練資料的題目模式，而不一定能泛化到 Kaggle 測試集。

此實驗說明：

較大的模型與較高的 validation accuracy 不一定代表較高的 Kaggle score。

7.3 Exp-3 : 改回 0.5B，降低 learning rate 並提高 dropout

Exp-3 改回 `Qwen2.5-0.5B-Instruct`，並調整訓練參數，包括降低 learning rate 與增加 dropout，以降低過擬合。

結果：

```
validation accuracy: 0.9223
Kaggle score: 0.66382
```

此組為目前最佳實驗結果。相較於 Exp-1，score 從 0.62127 提升到 0.66382。相較於 Exp-2，雖然 validation accuracy 較低，但 Kaggle score 更高。

此結果顯示，對於本任務而言，泛化能力比單純提高 validation accuracy 更重要。適當降低 learning rate 並提高 LoRA dropout，可以減少模型對訓練資料的過度記憶，使模型在測試集上表現更好。

7.4 Exp-4：使用 official + mixed data

Exp-4 使用官方訓練資料與補充資料 HW1_.csv 的混合資料進行訓練，希望透過增加資料量提升表現。

結果：

```
validation accuracy: 0.8864
Kaggle score: 0.63404
```

雖然 mixed data 增加了訓練資料量，但 Kaggle score 並沒有優於 Exp-3。根據資料觀察，補充資料中的答案分布偏向 B，而 D 的比例較少。這可能造成模型在測試時輸出答案分布偏移，導致 Kaggle score 下降。

因此，本實驗推測：

1. 額外資料不一定能提升成績。
2. 若補充資料分布與測試集不一致，可能造成負面影響。
3. 對選擇題任務而言，答案分布偏差會直接影響模型預測結果。

7.5 Exp-5：official data，增加 epoch 並降低 learning rate

Exp-5 回到官方資料，並以 Exp-3 為基礎繼續調整，增加 epoch 數並降低 learning rate，希望讓模型以更平滑的方式訓練更久。

結果：

```
validation accuracy: 0.9223
Kaggle score: 0.63829
```

此組的 validation accuracy 與 Exp-3 相同，但 Kaggle score 較低。這表示增加 epoch 並沒有帶來更好的泛化表現，反而可能使模型逐漸往訓練資料靠近。

此結果說明，即使 learning rate 較低，過多 epoch 仍可能造成模型對訓練資料過度適應。對於本任務而言，Exp-3 的 epoch 與正則化設定較為平衡。

8. 綜合討論

從五組實驗可以觀察到以下現象。

8.1 Validation accuracy 不等於 Kaggle score

Exp-2 的 validation accuracy 最高，達到 0.9873，但 Kaggle score 僅有 0.61276。相反地，Exp-3 的 validation accuracy 為 0.9223，但 Kaggle score 達到最高的 0.66382。

這表示本地 validation set 雖然可以用來檢查模型是否正常學習，但不能完全代表 Kaggle test set 的分布。若模型容量太大或訓練太久，validation accuracy 可能上升，但 Kaggle score 反而下降。

8.2 小模型可能比大模型更穩定

雖然 1.5B 模型具有更強表達能力，但在資料量有限的情況下，較大的模型更容易記住訓練資料。相較之下，0.5B 模型容量較小，在搭配適當 learning rate 與 dropout 時，反而有較好的泛化能力。

8.3 補充資料需要注意分布偏差

Mixed data 實驗顯示，增加資料量並不一定提升成績。如果補充資料的答案分布與官方資料不同，模型可能學到錯誤的 prior，例如偏向輸出某些選項，進而降低測試集表現。

8.4 正則化對此任務很重要

Exp-3 的成功說明，在小資料量 SFT 任務中，降低 learning rate 與增加 dropout 有助於改善泛化。此任務的目標不是讓模型完全記住訓練資料，而是學會可泛化到測試集的判斷方式。

9. 最佳實驗設定

目前最佳結果為 Exp-3：

```
Model: Qwen2.5-0.5B-Instruct
Data: official only
Epoch: 3
Learning rate: 5e-5
LoRA rank: 16
LoRA dropout: 0.10
Inference: logit-based scoring
Validation accuracy: 0.9223
Kaggle score: 0.66382
```

此組設定在模型容量、訓練強度與正則化之間取得較佳平衡，因此在 Kaggle public score 上表現最好。

10. 訓練過程心得

本次作業讓我熟悉了本地端大語言模型微調的完整流程，包括資料整理、模型載入、LoRA 訓練、validation 評估與 Kaggle submission 產生。相較於一般機器學習分類器，LLM SFT 的訓練流程需要處理更多工程細節，例如 tokenizer、prompt 格式、label masking、GPU 記憶體限制、套件版本相容性等。

在實驗過程中，我發現 validation accuracy 並不能完全代表 Kaggle score。起初我嘗試使用較大的 1.5B 模型並增加 epoch，雖然本地 validation accuracy 大幅提升，但 Kaggle 分數反而下降。這使我理解到，在資料量有限的任務中，模型容量過大或訓練過久都可能造成過擬合。

此外，我也觀察到資料分布的重要性。雖然加入補充資料可以增加訓練樣本數，但若補充資料的答案分布與官方資料不同，反而可能讓模型產生偏差。這說明資料品質與資料分布有時比資料數量更重要。

最後，透過比較不同設定，我認為本任務較適合使用較小模型搭配保守訓練參數，例如較低 learning rate、適中的 LoRA rank 與較高 dropout。這樣雖然不一定能得到最高的 validation accuracy，但更有機會在測試集上取得較好的泛化表現。

11. 未來改進方向

後續若要繼續提升分數，可以嘗試以下方向：

1. K-fold validation

目前 validation set 只使用單次 random split，可能無法穩定反映 test set 表現。未來可使用 K-fold 方式更可靠地評估模型。

2. 答案分布校正

觀察不同 submission 的 `A/B/C/D` 預測分布，若某選項過度偏高，可進一步調整推論階段的 logit bias 或資料抽樣策略。

3. 平衡補充資料

若使用 `HW1_.csv`，可以先對補充資料進行答案平衡抽樣，避免模型過度偏向 `B`。

4. 更細緻的 learning rate 搜尋

目前最佳為 `5e-5`，後續可嘗試 `3e-5`、`4e-5`、`6e-5` 等更細範圍搜尋。

5. 模型集成

可訓練多個不同 seed 或不同參數的模型，並在 logit 層級進行 ensemble，可能比單一模型更穩定。

6. Prompt 格式調整

可嘗試不同 prompt 模板，例如更明確要求模型只輸出單一英文字母，或調整題目與選項的排版方式。

12. 結論

本實驗完成了使用 Qwen 系列模型進行本地端 answer-only SFT 的完整流程。透過 LoRA / QLoRA，可在有限 GPU 資源下完成大型語言模型微調；透過 answer-only loss，模型能專注於學習正確答案；透過 logit-based inference，能穩定產生符合 Kaggle 格式的 `A/B/C/D` 輸出。

實驗結果顯示，最佳設定並非最大模型或最高 validation accuracy，而是具備較佳泛化能力的設定。目前最佳模型為 `Qwen2.5-0.5B-Instruct`，使用官方資料訓練 3 epoch，搭配 `lr=5e-5`、`lora_r=16` 與 `lora_dropout=0.10`，最終 Kaggle score 達到 `0.66382`。

本次實驗的主要收穫是：在小資料量 LLM SFT 任務中，避免過擬合、控制資料分布與選擇合適推論方式，往往比單純增加模型大小或提高 validation accuracy 更重要。